

ODB 编译与安装操作指南

文档类型： 技术部署部署文档 工具版本： build2 v0.18.1 / ODB v14.3.0 更新时间： 2026-07-07

本文档指导如何在 Linux 环境下通过 build2 构建系统从源码编译并安装 ODB Compiler（编译器）以及相关依赖运行时库。为了实现 ODB 编译器的完全自包含与跨环境独立运行，本方案要求将**整个完整的 GCC 14.3.0 工具链**直接嵌入部署到 ODB 的专属库路径下。

一、前期准备与环境说明

- **主构建目录：** 统一使用 /root/odb-build 作为源码下载与编译配置的工作空间。
- **目标安装路径：**
 - build2 系统组件 (b, bdep, bpkg, bx) 安装于 /usr/local/bin。
 - ODB Compiler 编译器及插件安装于 /root/odb-14.3.0。
 - libodb 相关运行时基础库与数据库扩展安装于 /root/libodb-14.3.0。

二、步骤一：编译与安装 build2 构建系统

ODB 的现代化编译链依赖 build2 系统的 bpkg 包管理器。首先必须完整编译并安装 build2 工具链。

```
mkdir -p /root/odb-build
cd /root/odb-build
curl -sSf0 https://download.build2.org/0.18.1/build2-install-0.18.1.sh
shasum -a 256 -b build2-install-0.18.1.sh # 验证脚本
sh build2-install-0.18.1.sh                # 执行安装
```

三、步骤二：构建与安装 ODB Compiler 并嵌入完整 GCC

此步骤将使用 bpkg 创建专用的编译器构建配置，并在安装完成后将整个 GCC 14.3.0 运行环境闭环嵌入到 ODB 内部。

1. 创建 ODB 构建配置环境

```
bpkg create -d odb-gcc-14.3.0 cc config.cxx=g++ config.cxx.coptions="-O3 -fPIC"
config.cxx.loptions="-static-libgcc -static-libstdc++" config.bin.rpath=.
config.bin.lib=static config.install.root=/root/odb-14.3.0
config.odb.plugins_config="../share/odb" config.odb.gxx_name="../lib/x86_64-linux-gnu/bin/g++"
config.install.sudo=sudo
```

2. 验证或修正 config.build 配置

如果需要确保 ODB 编译器在运行时通过相对路径正确寻找内置 GCC，可以进入配置目录验证：

```
cd odb-gcc-14.3.0/build
vim config.build
```

确保或手动添加以下行（对齐相对路径关系）：

```
config.odb.gxx_name = ../lib/x86_64-linux-gnu/bin/g++
```

保存后返回 odb-gcc-14.3.0/ 根目录。

3. 在线编译并安装 ODB 编译器

```
bpkg build odb@https://pkg.cppget.org/1/stable
bpkg install odb
```

4. 嵌入完整的 GCC-14.3.0 工具链（核心环节）

为了让 /root/odb-14.3.0/bin/odb 能够脱离宿主机环境正确运行，必须将**整个完整的 GCC-14.3.0 工具链目录结构**拷贝到 ODB 指定的共享库路径下，使其能够通过相对路径找到 bin/g++。

```
# 创建目标嵌入目录
mkdir -p /root/odb-14.3.0/lib/x86_64-linux-gnu/

# 将系统中已有的完整 GCC-14.3.0 工具链内容同步复制到该目录下
# (请根据实际环境中 GCC 14.3.0 的原始安装路径替换下面的 /usr/local/gcc-14.3.0/)
cp -a /usr/local/gcc-14.3.0/* /root/odb-14.3.0/lib/x86_64-linux-gnu/
```

💡 部署检查：

请务必确认执行后存在 /root/odb-14.3.0/lib/x86_64-linux-gnu/bin/g++ 这一完整路径。由于 ODB 编译器的内建检索逻辑被硬编码为相对路径 ../lib/x86_64-linux-gnu/bin/g++，只有整体嵌入完整的二进制、依赖库和前端，才能保障其自包含的稳定性和完美运行。

四、步骤三：构建与安装 libodb 及其数据库依赖库

接下来并行编译 C++ 项目运行时所需的运行时基础库与数据库插件，并安装到独立的目录中。

1. 创建运行时库构建配置环境

```
cd /root/odb-build
bpkg create -d libodb-gcc-14.3.0 cc config.cxx=g++ config.cc.coptions="-O3 -fPIC" config.bin.rpath=. config.bin.lib=static config.install.root=/root/libodb-14.3.0 config.install.sudo=sudo
```

2. 添加仓库源并拉取元数据

```
cd libodb-gcc-14.3.0/  
bpkg add https://pkg.cppget.org/1/stable  
bpkg fetch
```

3. 批量编译与全局递归安装

```
bpkg build libodb          # 基础核心库  
bpkg build libodb-sqlite  # SQLite 数据库插件  
bpkg build libodb-pgsql   # PostgreSQL 数据库插件  
bpkg build libodb-boost   # Boost 类型支持扩展  
  
# 执行全局递归安装到 /root/libodb-14.3.0  
bpkg install --all --recursive
```

五、验证验证

执行以下命令验证 ODB 编译器是否可以闭环定位并正常输出版本信息：

```
/root/odb-14.3.0/bin/odb --version
```